

Chapter 1

Mission Bus Integration Lab (MBIL)

1.1 Project purpose

Mission Bus Integration Lab (MBIL) is a deterministic, configuration-driven software system that simulates a simplified aerospace mission environment built around a central data bus, redundant mission computers, subsystem messaging, defined failure modes, observability, and failover behavior.

The goal is not to make a flashy simulator

The goal to demonstrate that I understand how **mission-critical systems behave, fail, recover, and get debugged.**

1.2 What MBIL should communicate

When someone sees this project, they should think:

- ★ This person understands reliable system design.
- This person thinks in interfaces and contracts
- This person knows how to model faults and degraded operation
- This person knows how to make systems observable
- This person can separate control logic, transport, config, and monitoring

MBIL should read like a **systems engineering lab**, not a game and not a toy simulation

1.3 Core project statement

MBIL is a **tick-driven mission-system simulation platform** with these defining properties:

- Deterministic execution
- Configuration-defined topology
- Redundant control nodes
- First-class failure scenarios

- structured observability
 - Recoverable degraded operation
-

1.4 Design priorities

These are the real priorities, in order:

Priority 1 – Determinism The same scenario should produce the same behavior every run unless randomness is explicitly enabled

Priority 2 – Reliability modeling Failures are not side noes. They are part of the system design.

Priority 3 – Observability Every important decision, state transition, and message path should be visible in logs or snapshots.

Priority 4 – Clear modular architecture Subsystem logic, bus behavior, timing, fault injection, and login must stay cleanly separated.

Priority 5 – Configurability System topology and scenarios should be changel from config files, not by rewriting core logic.

1.5 Function Goals

MBIL must support:

- Subsystem-to-subsystem communication through a central bus
 - Multiple sybsystem types
 - Dual mission computer architecture
 - Heartbeat monitoring
 - Failover behavior
 - Defined failure injection
 - Message routing
 - Deterministic Scheduling
 - Structured logs
 - State Snapshots
 - Scenario-based runs
-

1.6 High-level Architecture

MBIL should be split into these major layers. **A. Simulation Core**

Responsible for:

- Tick loop
- Execution order
- Simulation Duration
- System lifecycle
- Calling each subsystem in deterministic order

B. Bus Layer Responsible for:

- Receiving Outgoing messages
- Storing pending messages
- Applying routing rules
- Applying message-related faults
- Delivering ready messages
- Preserving deterministic ordering

C. Subsystem Layer Responsible for:

- Mission Computers
- Sensors
- Display / Endpoints
- Health State
- Behavior per tick
- Message Handling

D. Fault Management Layer Responsible for:

- Defined Failure modes
- Scripted Scenario Events
- Optional Random Fault behavior later
- Recording Fault Activations
- Applying Degradation Cleanly

E. Observability Layer Responsible for:

- Structured Event Logging
- Traceable Message Lifecycle
- State Snapshots

- Failover Reportin
- Auditability

F. Configuration Layer Responsible for:

- Loading JSON config
 - Validating Topology
 - Instantiating Subsystem Objects
 - Loading Scenario/Fault Definitions
 - Setting timing and thresholds
-

1.7 Runtime Model

MBIL should use a **fixed tick** simulation model.

Example:

- One tick every 100 ms simulated time
- Run for 300 ticks per scenario

This is important because it gives you:

- Deterministic Behavior
- Clean Testability
- Easier Fault scripting
- Easier replay and debugging

Tick Processing Order

Use a Fixed order every tick:

Phase 1 – Scenario updates

- Activate any scheduled Failures
- Deactivate expired Failures
- Apply Manual state changes if configured

Phase 2 – Sensor Updates

- Sensors decide whether to emit new data this tick

Phase 3 – Mission Computer Updates

- Active controller processes a state
- Standby Controller monitors health
- Heartbeats generated if healthy

Phase 4 – Endpoint / Display Updates

- Display or logger consumers update their local state.

Phase 5 – Bus Processing

- Pending messages sorted and processed
- Faults applied to eligible messages
- Delayed messages retained
- Deliverable messages routed

Phase 6 – Observability Output

- Event logs Written
- Snapshot Optionally captured

That order should be documented and never casually changed.

1.8 Deterministic behavior rules

This is now a first-class design feature.

MBIL determinism rules Rule 1: fixed tick rate The simulation advances in discrete ticks.

Rule 2: stable subsystem order Subsystems are always ticked in the same deterministic order, such as sorted by ID.

Rule 3: Stable message ordering Messages created in the same tick are delivered in a defined order:

- Priority
- Tick created
- Source ID
- Message sequence number

Rule 4: No hidden asynchronous behavior in MVP No real multithreading in the initial version.

Rule 5: Reproducible scenario results A scenario file should produce identical output unless randomness is explicitly enabled with a fixed seed. This matters because it makes the system feel like an engineering lab rather than an app demo.

1.9 Core subsystem model

Each subsystem should implement a common interface.

- `id()`
- `type()`
- `tick(context)`
- `receive(message)`
- `health()`
- `snapshot()`

Health states

Every subsystem should have explicit health states such as:

- Nominal
- Degraded
- Offline
- Failed
- Recovering

This makes failure handling more professional and easier to test.

1.10 Main subsystem types

A. Mission Computer

Two instances:

- **MC1** = primary
- **MC2** = backup

Responsibilities:

- Process incoming sensor/status messages
- Emit heartbeats
- Generate command/status output
- Maintain own health state
- Participate in failover logic

MC1 Behavior

- Starts as active controller
- Processes sensor inputs
- Emits heartbeat periodically
- Produces system status and commands

MC2 Behavior

- Starts in standby
 - Watches for MC1 heartbeat loss
 - Tracks timeout threshold
 - Assumes control when failover condition is met
 - Announces failover in log/event stream
-

B. Sensors

Example Sensors

- Altitude Sensor
- Temperature Sensor
- Navigation Source

Responsibilities

- Emit data on configured interval
- Send structured sensor messages
- Support degraded or failed behavior under fault scenarios

Sensor outputs should be simple but believable:

- Value
 - Valid Flag
 - Quality / State
-

C. Display/Endpoint

Responsibilities:

- Receive commands/status
- Display system state through logs or summary output
- Report currently active mission computer
- Show degraded conditions

This is not a GUI priority. Console or structured log output is enough.

1.11 Message bus design

The bus is central, but should stay simple.

Bus responsibilities

- Accept outgoing messages from subsystems
- Assign message metadata if needed
- Hold pending and delayed queues
- Apply failure rules
- Route messages by destination
- Support broadcast
- log delivery outcome

Message routing modes

Support:

- point-to-point
 - broadcast
 - optional group routing later
-

1.12 Message model

Keep the message model explicit and interview-friendly.

Message fields

Each message should include:

- message_id
- sequence_number
- tick_created
- tick_due
- source_id
- destination_id
- message_type
- priority
- payload
- trace_id or correlation ID

That last field is very good for observability.

Message types

Starts with:

- HEARTBEAT

- SENSOR_DATA
- STATUS
- COMMAND
- ALERT
- FAILOVER_NOTICE

Payload model

Use a simple tagged payload design.

Examples:

- Heartbeat payload: current role, health
 - Sensor payload: sensor name, numeric value, validity
 - Status payload: active controller, mode, health summary
 - Command payload: command name, parameter
-

1.13 Failure modes as a first-class feature

This is one of the the biggest improvements.

Do not describe this as "supports faults."

Describe it as:

MBIL includes defined operational failure modes and corresponding system responses.

Core failure mode categories

1. Bus timeout / delivery failure

A Message does not arrive in expected time.

Effect:

- Receiving system may mark source stale
- Failover logic may be triggered if heartbeats are affected

3. Message delay

Message is delivered after configured delay.

Effect:

- Stale data
- Delayed heartbeat detection
- Timing edge cases

4. Message corruption

Payload is invalid or altered.

Effect:

- Receiving system rejects or flags data
- Degraded mode may be entered

5. Node failure

Subsystem goes offline.

Examples:

- MC1 fails
- Sensor fails

- Display goes offline

6. Node degradation

Subsystem still runs but performs poorly.

Examples:

- Lower sensor update rate
 - Heartbeat jitter
 - Intermittent bad data
-